

ACCOUNTING FOR SIMULATOR BRANCHING IN PREDICTIVE REPRESENTATION LEARNING

Internal working draft v0, revision 2 (with 2×2 results) — not submitted to any venue

Anonymous authors

Paper under double-blind review

ABSTRACT

Action-conditional predictive objectives are a standard way to train history representations in partially observable environments, and a resettable simulator makes it tempting to supervise them with the outcomes of several alternative actions taken from the same saved state. Such branching consumes environment transitions and relies on a privilege, state restore, that ordinary interaction does not provide. We study how to account for this privilege when comparing representation learners. We implement a small aliased T-maze in which the current observation at a junction is identical for both values of an earlier clue, a transition ledger that charges every main-trajectory, branch and evaluation step against one hard cap and counts restores separately, and a fixed readout protocol that trains a fresh action-conditional reward head on a frozen encoder and scores a model-predictive planner by its junction decision (forced-commit accuracy). An evaluator diagnostic with fixed encoders shows that this readout can separate informative from uninformative histories: a hand-designed history-only control reaches forced-commit accuracy 1.000 in all three diagnostic seeds, while an untrained recurrent encoder and a memoryless control both stay at 0.500, leaving a headroom of 0.500. In the same runs, the full-action planning success of the two clue-blind encoders differs only through stalling behaviour, so we do not use it to compare representations. We then ran a 2×2 study that crosses action conditioning (an action-conditional predictor versus an action-free ablation) with the collection method (branching versus no branching). Each collection method had 10,000 pretraining transitions per seed, and every encoder was read out through a separate restore-free pool. The readout again recognised the informative control, at forced-commit accuracy 1.000 in all seeds. None of the four learned encoders exceeded the untrained encoder: their accuracies ranged from 0.475 to 0.500, so the pre-registered learning-benefit contrast failed (`LEARNING_BENEFIT_NOT_SUPPORTED`). Because every learned arm stayed at chance, the collection and conditioning contrasts sit at a floor and do not separate the two effects; we do not read them as evidence of no effect. We make no claim that low prediction error guarantees planning for arbitrary policies, and all results come from one synthetic task family with three seeds.

1 INTRODUCTION

In a partially observable environment the current observation can be identical in situations that call for different actions. This is perceptual aliasing (Whitehead & Ballard, 1991; McCallum, 1996). An agent must then act on a summary of its history. Predictive state representations make the point precise: a history is summarised by predictions of *action-conditional* tests, that is, of future observations given future actions (Littman et al., 2001; Singh et al., 2004). Neural descendants of this idea train recurrent history encoders with multi-step, action-conditional prediction losses (Gregor et al., 2019; Guo et al., 2018; 2020; Kwon et al., 2024). Related latent-consistency objectives with action-conditional dynamics are standard in model-based reinforcement learning (Schwarzer et al., 2021; Hansen et al., 2024; Schrittwieser et al., 2020).

An action-conditional target is only observed for the actions that were actually taken. With a resettable simulator one can do more: save the complete simulator state, try several first actions from it, and restore the state between attempts. This *branching* yields paired counterfactual targets from the same history. It is an old device, used for instance by the vine procedure of TRPO (Schulman et al., 2015) and by PSR learning with resets (James & Singh, 2004). Branching is not free, however. Each branch step is an environment transition, and the restore operation is a privilege that ordinary interaction with a real system does not provide. Comparisons of representation learners that ignore either cost can credit an objective with gains that actually come from extra experience or from simulator access.

This draft asks a narrow question: *under an equal environment-transition budget and a fixed readout protocol, how do the effects of branching collection and of action conditioning separate?* We do not propose a new objective. Action-conditional prediction, multi-step prediction, recurrent world models, and state-restore branching are all prior work (Section 2). Our contributions are limited to the following.

- **An accounting protocol and implementation.** A single transition ledger charges every main-trajectory, branch, and evaluation step against one hard cap. Snapshots and restores are counted separately and are forbidden during evaluation. Actors and learners receive only the observable history, never simulator state or evaluator labels. These properties are checked by unit tests. The tests are technical checks, not evidence about any method.
- **An evaluator diagnostic.** Before comparing learned encoders, we check that the readout (a frozen encoder, a fresh action-conditional reward head, and an exhaustive model-predictive planner) can separate informative from uninformative representations. It can: a hand-designed history-only control and an untrained recurrent encoder differ by 0.500 in forced-commit accuracy in every diagnostic seed (Section 5.1).
- **A small controlled 2×2 study** that crosses action conditioning with branching under an equal transition budget and reports experience and compute separately (Section 5.2). Its outcome is negative. Under the tested budget and fixed pretraining settings, no learned encoder exceeded an untrained one in forced-commit accuracy. The study therefore does not separate the effect of branching from that of action conditioning.

We also report what the evidence does not show. The primary metric, forced-commit accuracy, measures a single junction decision, not planning success in general. The full-action planner’s success rate is confounded by stalling behaviour. All results come from one synthetic task family with three seeds, and the comparison with an external action-conditioned method such as that of Kwon et al. (2024) is not yet run.

2 RELATED WORK

Predictive state and history representations. PSRs represent state as predictions of action-conditional tests (Littman et al., 2001; Singh et al., 2004). Temporal-difference networks learn such predictions from the actions actually taken (Sutton & Tanner, 2004). Predictive-state decoders and PSRNNs attach future-observation prediction to recurrent networks; in the formulations we checked, the targets are not conditioned on future actions (Venkatraman et al., 2017; Downey et al., 2017), which makes them structurally closest to our action-free ablation. The approximate information state framework bounds the planning loss of a history compression that predicts rewards and its own next value given an action (Subramanian et al., 2022). Ni et al. (2024) relate one-step self-predictive history objectives to that framework.

Action-conditional multi-step prediction. SimCore rolls a recurrent belief forward conditioned only on the agent’s actions (Gregor et al., 2019). CPC|Action and PBL use action-conditional multi-step predictions from compressed histories (Guo et al., 2018; 2020). Guo et al. (2018) report that multi-step prediction and action conditioning are critical for their belief representations; our action-conditional versus action-free contrast is therefore not a new question. Multi-step latent prediction conditioned on actions also underlies SPR (Schwarzer et al., 2021), TD-MPC2 (Hansen et al., 2024), MuZero (Schrittwieser et al., 2020), and earlier action-conditional video prediction (Oh et al., 2015). Recurrent world models such as PlaNet and DreamerV3 learn latent dynamics in POMDPs (Hafner

et al., 2019; 2023). Khetarpal et al. (2025) compare per-action and policy-marginal self-prediction theoretically in MDPs. Task-oriented alternatives include bisimulation metrics (Zhang et al., 2021), value equivalence (Grimm et al., 2020), and successor features (Dayan, 1993; Barreto et al., 2017).

Closest work. Kwon et al. (2024) train a history encoder with an action-conditional k -step observation predictor, decouple representation learning from reinforcement learning, and evaluate on memory tasks. Their test actions are inserted once per episode along the executed trajectory, without simulator branching. In the version we read, we did not find an action-free ablation. Our action-conditional variant is a small re-implementation in the same spirit, not an official reproduction of their method.

Resets, branching, and memory tasks. State restore for branching rollouts appears in TRPO’s vine procedure (Schulman et al., 2015). Reset-and-replay is used to estimate PSR tests from a common history (James & Singh, 2004), and local simulator access has been analysed theoretically (Mhammedi et al., 2024). In our deterministic task, reset-and-replay and save-and-restore are nearly equivalent. T-maze memory tasks and recurrent or world-model baselines for them are studied by Ni et al. (2022), Ni et al. (2023), and Samsami et al. (2024).

3 PROBLEM AND EXPERIMENTAL SETTING

Aliased T-maze. An episode starts in a clue cell whose observation reveals a hidden binary clue $c \in \{A, B\}$. The agent then walks a corridor of length L , sampled uniformly from $\{3, \dots, 6\}$. Corridor observations are drawn from 4 distractor symbols independently of c . The corridor ends at a junction whose observation is identical for both clues. The actions are FORWARD, LEFT, and RIGHT. In the corridor, LEFT and RIGHT leave the agent in place; at the junction, FORWARD leaves the agent in place, and LEFT or RIGHT ends the episode in an arm. In the symmetric variant used throughout, the arm that matches the clue gives a goal observation and reward $+1$, and the other arm gives reward -1 . Episodes time out after $L + 2 + 6$ steps. Two histories that differ only in the clue therefore share the current observation at the junction, yet the consequence of LEFT differs between them. Under any clue-independent distribution over action sequences, the action-marginal distribution of future observations and rewards is identical for both clues. Our tests confirm this by exact enumeration for horizons up to three, starting at the junction and one step before it, and also confirm that the action-conditional futures differ. This is a direct instance of the PSR observation that the relevant predictions are action-conditional (Littman et al., 2001; Singh et al., 2004), and we do not present it as a new result.

Information access. The simulator’s complete state is private: clue, corridor length, position, and random-number-generator state. It is available only to data collectors (for snapshot and restore) and to the evaluator (for labels). Actors, encoders, prediction heads, and the planner receive only the observable history $h_t = (o_0, a_0, r_1, \dots, a_{t-1}, r_t, o_t)$. Ground-truth labels are returned in a separate structure that training code never receives. Unit tests check the relevant function signatures, the whitelisted record fields, that serialized datasets contain no hidden fields, and that the encoder output at time t is unaffected by later inputs.

Transition budget and the restore privilege. Every environment step, whether on the main trajectory, in a branch rollout, or during evaluation, is charged to a ledger *before* it executes. A step that would exceed the cap is not executed. Data collection spends the budget exactly, so arms with equal caps also have equal *actual* transition spend. Snapshots and restores are not transitions, but they are a simulator privilege; the ledger counts them separately, and the evaluation ledger forbids them. An equal transition budget does not imply equal compute, and we report arithmetic cost (multiply-accumulates, MAC) and CPU time separately.

Branching collection. Main trajectories follow a history-based behaviour policy. It is uniform at the junction and presses FORWARD with probability 0.8 elsewhere. At each main step, with probability 0.25, the collector snapshots the simulator state. For each of the three first actions it then rolls out up to $k = 3$ steps with a uniform continuation policy, restoring the snapshot between rollouts and after the last one. Random streams are separated so that branching never changes the

main trajectory; a test checks this property. A branch set is started only if the remaining budget can cover it.

Readout protocol. Representations are compared through one fixed downstream path. The encoder is frozen. Its features are z-scored per dimension, with statistics fitted only on the readout training data. A fresh action-conditional reward head, a latent rollout with hidden size 16, is trained with squared reward error at learning rate 0.01. The head initialization, data, and optimizer are identical across encoders. We keep the checkpoints after 20/40/80 epochs of a single run and select the one with the lowest development reward error; the test split is never used for selection. A model-predictive planner enumerates all action sequences of length 2 and scores them by discounted predicted reward ($\gamma = 0.9$).

Metrics. Each test episode starts with a scripted, observation-only prefix that presses FORWARD until the junction observation appears. *Forced-commit accuracy*, the primary metric, is the fraction of test episodes in which the planner, restricted to first actions LEFT and RIGHT at the first junction arrival, prefers the optimal arm. It measures one aliased decision and is not a planning success rate. The *full-action* planner then acts with all three actions until the episode ends. From it we report success, mean return, and timeout rate, together with the number of exact forced-commit ties. The evaluator assigns clues by alternating episode index, so every constant decision rule scores exactly 0.5. A logistic probe of the clue on the junction features is fitted on a separate calibration split. We use it only as a diagnostic: failure of a linear probe does not show that the features contain no clue information.

4 VARIANTS UNDER STUDY

All learned encoders are Elman recurrent networks with tanh units and hidden size 16. The input at time t is the one-hot observation o_t , the one-hot previous action, and the previous reward, and the recurrence is $z_t = \tanh(W[x_t; z_{t-1}] + b)$ with $z_{-1} = 0$. The predictor is a latent rollout $u_0 = z_t$, $u_{j+1} = \tanh(W_d[u_j; \alpha_j] + b_d)$, with linear heads for observation logits and reward. Pretraining minimizes the cross-entropy of o_{t+j+1} plus the squared error of r_{t+j+1} for $j < k$. The loss is taken over all k -step windows of the main trajectories and over all branch rollouts. After a termination, targets are an absorbing padding symbol with reward zero and remain supervised, because the end of an episode is observable. After a truncation (timeout or budget), targets are masked. The variants differ only in α_j and in the pretraining data.

- **ACP (action-conditional prediction):** α_j is the one-hot action a_{t+j} . The objective fits $p(o_{t+j+1}, r_{t+j+1} \mid h_t, a_{t:t+j})$. This conditional target is unconfounded only if the actions in the data do not depend on hidden state. Here the behaviour policy reads only the history, and the continuation policy is uniform.
- **MSP (action-free multi-step prediction):** $\alpha_j = 0$. The objective fits the action-marginal future under the action distribution of the data, so it depends on the behaviour policy. MSP is an *ablation* of ACP. It has the same nominal parameter count, but the action columns of W_d receive no gradient, and we do not treat it as a strong or state-of-the-art baseline.
- **Collection:** *branching* data or *no-branching* data under the same transition cap. The difference between them is a difference in collection method, not a controlled comparison on identical data.
- **Reference encoders:** an *untrained* recurrent encoder with the same initialization as the learned encoders; a hand-designed *history-only informative control* whose features are the clue identity read from o_0 together with the one-hot current observation; and, in the diagnostic only, a *memoryless control* that sees only the one-hot current observation. The informative control is a reference point for the readout, not an upper bound on what a learned encoder can achieve.

Low prediction error is not claimed to guarantee planning performance. The readout and planner above cover one decision, horizon 2, and the action support present in the data. Nothing here speaks to other policies, longer horizons, or states outside the data’s coverage.

Table 1: Evaluator diagnostic (fixed encoders, symmetric maze). Entries are per seed (1000, 1001, 1002). Forced-commit is the primary junction-decision metric. Full-action success and timeout come from the unrestricted planner. “Epochs” is the head checkpoint selected on development reward error. Source: `runs/diag_20260926T160006Z_281020c`.

Encoder	Forced-commit	Full-action success	Timeout	Probe (test)	Epochs
History-only informative control	1.000 / 1.000 / 1.000	1.000 / 1.000 / 1.000	0.000 / 0.000 / 0.000	1.00 / 1.00 / 1.00	80 / 80 / 40
Untrained recurrent encoder	0.500 / 0.500 / 0.500	0.500 / 0.500 / 0.500	0.000 / 0.000 / 0.000	0.47 / 0.53 / 0.57	20 / 20 / 80
Memoryless control	0.500 / 0.500 / 0.500	0.500 / 0.000 / 0.000	0.000 / 1.000 / 1.000	0.50 / 0.50 / 0.50	80 / 80 / 80

5 EXPERIMENTS AND OBSERVATIONS

5.1 EVALUATOR DIAGNOSTIC WITH FIXED ENCODERS

Question and design. Before comparing learned encoders, we asked whether the readout of §3 can separate informative from uninformative histories at all. If it cannot, a learned encoder cannot be credited with anything. Three fixed encoders went through the identical readout: the history-only informative control, the untrained recurrent encoder, and the memoryless control. The diagnostic used 10,000 readout-training transitions (collected with branching), 2,000 development transitions, 200 calibration episodes, and 200 clue-balanced test episodes per seed. Seeds 1000, 1001, 1002 were reserved for this diagnostic. The decision rules were fixed before the run. The evaluator is unresolved if the informative control falls below 0.90 or the memoryless control deviates from 0.5 by more than 0.10 in any seed. There is headroom if the informative control exceeds the untrained encoder by at least the minimum effect of interest (0.10) in every seed.

Observations. Table 1 lists the per-seed results. All 9 of 9 cells completed, and the verdict is `HEADROOM_PRESENT`. The informative control reached forced-commit accuracy 1.000 in every seed. The untrained recurrent encoder and the memoryless control both stayed at exactly 0.500. Each chose the same arm in every test episode, which on a clue-balanced test set gives exactly one half. A leak of episode index or of the balancing pattern into the inputs would have shown up as above-chance accuracy for these clue-blind encoders, and it did not. The linear clue probe on the untrained encoder’s junction features scored between 0.47 and 0.57 on test episodes. The readout could not extract the clue from these features, but that does not show that the features carry no clue information.

The full-action metric is confounded. Full-action success separates the two clue-blind encoders, for which forced-commit accuracy is identical. The untrained encoder committed to an arm (success 0.500). The memoryless control, in two seeds, preferred `FORWARD` at the junction until timeout (timeout rate up to 1.000). The difference reflects commit-versus-stall behaviour of a model-predictive planner with an uninformative head, not the clue content of the representation. We therefore report full-action metrics but do not base comparisons on them.

Cost. The diagnostic used 53,503 environment transitions in total. Of these, 10,103 main and 19,897 branch steps were readout-training data, with 9,888 restores. Test episodes used 14,217 transitions and no restores. The run took 1,189 CPU seconds and 13.17 GMAC in a single process with peak resident memory 26.9 MB (commit 281020c). The readout-training data in this diagnostic were collected with branching, so this readout was *not* restore-free. The 2×2 study below uses a separate restore-free readout pool.

5.2 2×2 STUDY: ACTION CONDITIONING $\times$ BRANCHING

Design. Six arms were evaluated with seeds 1, 2, 3 in the symmetric maze: A, ACP with branching; B, ACP without branching; C, MSP with branching; D, MSP without branching; E, the untrained recurrent encoder; and F, the history-only informative control. These seeds are the first three of a previously registered evaluation list whose evaluation outputs had not been viewed. They are new seeds of the same design, not an independent replication. Each collection method spent exactly 10,000 environment transitions per seed on pretraining, with branch steps included for the branching data. Arms A and C trained on one shared branching dataset, and B and D on one shared no-

Table 2: 2×2 study, per seed (1, 2, 3). The primary metric is forced-commit accuracy, a junction decision rather than a planning success rate. The probe is a logistic probe fitted on the calibration split and scored on test junction features. “Head epochs” is the checkpoint selected on development reward error; the grid was not extended when 80 was selected. Source: runs/study2x2_20260926T225310Z_fd781be.

Arm	Forced-commit	Probe (test)	Head epochs	Status
A: ACP + branching	0.500 / 0.500 / 0.475	0.65 / 0.53 / 0.75	20 / 20 / 20	OK
B: ACP + no branching	0.500 / 0.500 / 0.500	0.53 / 0.64 / 0.54	20 / 20 / 20	OK
C: MSP + branching	0.500 / 0.500 / 0.500	0.61 / 0.53 / 0.59	80 / 20 / 20	OK
D: MSP + no branching	0.500 / 0.500 / 0.500	0.56 / 0.51 / 0.62	20 / 20 / 20	OK
E: untrained recurrent	0.500 / 0.500 / 0.500	0.56 / 0.52 / 0.52	20 / 20 / 20	OK
F: informative control	1.000 / 1.000 / 1.000	1.00 / 1.00 / 1.00	20 / 40 / 20	OK

branching dataset. The comparison between A and B, or between C and D, is therefore a comparison of collection methods at equal transition cost, not a comparison on identical data. Pretraining used 20 epochs at learning rate 0.01. These settings were taken from an earlier plan and were not tuned in this study. All six arms were read out through the same restore-free pool, collected once per seed without branching: 10,000 training and 2,000 development transitions. Each arm’s feature standardization was fitted on its own features of the training pool. Head checkpoints at 20/40/80 epochs were selected on development reward error, and 200 clue-balanced test episodes were used. The decision rules were fixed before the run: the positive control had to reach at least 0.90; learning benefit required $ACP_b - \text{untrained} \geq 0.10$ in every seed; saturation meant that all learned arms were at least 0.95; and a collection or conditioning effect required the corresponding pre-registered contrast to be at least 0.10 in every seed. The interaction and the non-registered contrasts are reported descriptively.

Accounting and splits. Across the three seeds, pretraining consumed 60,000 transitions and 10,048 restores, all of them in the branching datasets. The readout pool consumed 36,000 transitions and 0 restores. Testing consumed 31,470 transitions with 0 restores. Table 5 in the appendix gives the per-seed ledgers. The five splits share no latent episode seeds (0 overlaps). Branch rollouts stay inside their root episodes. By design, the branching and no-branching pretraining sets share their first 393–399 root episodes, since branching does not alter main trajectories. Seed separation does not imply content separation: between 0.10 and 0.29 of the test episodes’ junction-prefix observation sequences also occur in a pretraining or readout set.

Observations. All 18 of 18 cells completed (Table 2), and the pre-registered decision rule returned `LEARNING_BENEFIT_NOT_SUPPORTED`. The informative control again reached forced-commit accuracy 1.000 in every seed, so the readout was able to exploit clue information when it was cleanly present. The four learned encoders scored between 0.475 and 0.500, and the untrained encoder scored 0.500. The pre-registered learning-benefit contrast $ACP_b - \text{untrained}$ was 0.000, 0.000, and -0.025 in the three seeds. Every contrast in Table 3 lies between -0.025 and 0.000. These values do not show that branching and action conditioning have no effect. Every learned arm is at the chance floor of this readout, so the contrasts cannot separate the two effects. Pretraining itself reduced the prediction loss of every learned arm (Table 7). Linear probes of the learned encoders’ junction features scored between 0.51 and 0.75 on test episodes, compared with 0.52–0.56 for the untrained encoder. Some learned features may therefore carry partially decodable clue information that the reward head did not turn into decisions; with 200 probe episodes per cell, we treat this as an observation to examine, not a finding. As in the diagnostic, the full-action metrics varied with commit-versus-stall behaviour, with timeout rates up to 1.000 for arms whose forced-commit accuracy was at chance (Table 6).

Equal transitions are not equal compute. At the same transition cap, the branching datasets contain fewer episodes and fewer prediction windows. Pretraining on them therefore cost roughly half the multiply-accumulates and CPU time of the no-branching datasets (Table 4). Readout cost was similar across arms, because all arms shared the readout pool. Test transitions differ between arms because stalling episodes run until timeout. The study took 5,202 CPU seconds and 56.15 GMAC, with peak resident memory 42.0 MB, in a single process under a hard cap of 7,000 CPU seconds (commit fd781be).

Table 3: Paired forced-commit contrasts per seed. The minimum effect of interest (0.10) is applied only to the four contrasts that were registered with it before the study; the others are descriptive (“n/a”). A value of zero at the chance floor is not evidence of equivalence.

Paired contrast (forced-commit)	Per seed	Mean	$\geq$ MIE in all seeds
$ACP_b - ACP_n$ (collection, ACP)	0.000 / 0.000 / -0.025	-0.008	no
$MSP_b - MSP_n$ (collection, MSP)	0.000 / 0.000 / 0.000	0.000	n/a
$ACP_b - MSP_b$ (conditioning, branch data)	0.000 / 0.000 / -0.025	-0.008	no
$ACP_n - MSP_n$ (conditioning, no-branch data)	0.000 / 0.000 / 0.000	0.000	no
$(ACP_b - MSP_b) - (ACP_n - MSP_n)$	0.000 / 0.000 / -0.025	-0.008	n/a
ACP_b - untrained	0.000 / 0.000 / -0.025	-0.008	no
ACP_n - untrained	0.000 / 0.000 / 0.000	0.000	n/a
MSP_b - untrained	0.000 / 0.000 / 0.000	0.000	n/a
MSP_n - untrained	0.000 / 0.000 / 0.000	0.000	n/a

Table 4: Compute and test experience per arm (range over seeds). Pretraining uses the same transition cap for branching and no-branching data, but not the same compute.

Arm	Pretrain CPU (s)	Pretrain GMAC	Readout CPU (s)	Test transitions
A: ACP + branching	56–58	0.62	235–238	1,311–2,167
B: ACP + no branching	105–108	1.19	233–242	1,312–2,511
C: MSP + branching	57–60	0.62	234–242	1,273–2,511
D: MSP + no branching	104–107	1.19	231–234	1,273–2,511
E: untrained recurrent	0	0.00	230–234	1,312–2,511
F: informative control	0	0.00	229–236	1,273–1,312

6 DISCUSSION AND LIMITATIONS

What the current evidence supports. The accounting and information-access properties hold in our implementation, as checked by unit tests. These are technical checks, not evidence about any method. In two separate runs with different seeds and readout pools, the fixed readout separated a hand-designed history-only control (forced-commit accuracy 1.000 and 1.000) from the untrained encoder (0.500 and 0.500). One run used a readout pool collected with branching, the other a restore-free pool. The evaluation can therefore detect clue information when it is cleanly present. In the 2×2 study, however, none of the learned encoders reached that point. Their decisions stayed at the untrained encoder’s chance level, so the study gives no evidence for either a collection benefit or an action-conditioning benefit. It also gives no evidence against either, because the contrasts sit at a floor. The only quantitative statement the study supports is conditional: with 10,000 pretraining transitions, 20 epochs at learning rate 0.01, and this readout, neither objective nor collection method produced a usable clue representation in three seeds.

Possible explanations (CONJECTURE, not tested). The following explanations are consistent with the observations. None has been tested, and we did not change the environment or the settings after seeing the results.

1. The clue must be carried across the whole corridor, with distractors, while the k -step prediction targets depend on it only for windows that start within a few steps of the junction. The training signal for retaining the clue may therefore be weak for both objectives.
2. The fixed pretraining budget and schedule, taken from an earlier plan without tuning, may be too short for backpropagation through time to establish the long-range dependency.
3. The reward head may be insensitive to weak clue features. Linear probes reached up to 0.75 on some learned encoders while forced-commit accuracy stayed at chance.

Distinguishing these explanations would require new runs, which we list as not run in Appendix D.

Limitations.

- **One synthetic task family.** All results come from a small deterministic aliased T-maze. The extreme values of 0.5 and 1.0 reflect this task and should not be expected elsewhere.
- **Reset privilege.** Branching requires a simulator that can save and restore arbitrary states. Any advantage of branching collection would be conditional on that privilege and would not transfer to systems that lack it. Our readout pool is restore-free, but the branching pretraining is not.
- **Three seeds.** Each comparison uses three seeds of one configuration. These are repetitions under a fixed design, not independent replications. A non-significant or null contrast, especially at a floor, is not evidence of equivalence.
- **Forced-commit primary metric.** The primary metric scores one aliased decision after a scripted prefix. It is not a planning success rate, and we make no claim about other policies, horizons, or states outside the data’s coverage. Low prediction loss (Table 7) did not translate into better decisions here, and in general it is not a guarantee of planning performance.
- **Head-dependent readout.** Conclusions are conditional on the frozen-encoder readout: feature standardization, a small reward head, three epoch candidates, and a depth-2 planner. A different readout could rank encoders differently. The probe is linear, and its failure is not a proof of absent information.
- **Weak comparison set.** The action-free ablation is not a strong baseline, and our action-conditional variant is not an official reproduction of any published method. We have not compared against an external action-conditioned method such as that of Kwon et al. (2024), or against recurrent world models (Hafner et al., 2023; Samsami et al., 2024).
- **Untuned pretraining.** The pretraining settings were fixed in advance and not tuned. The negative result may be specific to them.
- **Protocol changes after early looks.** Several protocol choices were revised after smoke runs and control outputs but before any comparison of learned encoders (Appendix B).

7 CONCLUSION

Comparisons of predictive representation learners that use simulator branching should charge every branch step to the same transition budget as ordinary interaction. They should count restores as a separate privilege, report compute separately from experience, and read representations out through a restore-free pool. We implemented such an accounting in a small aliased T-maze. We showed that a fixed frozen-encoder readout detects cleanly available clue information, and that full-action success is confounded by stalling. In a 2×2 study at 10,000 pretraining transitions per collection method, neither action-conditional nor action-free pretraining, with or without branching, improved forced-commit accuracy over an untrained encoder in any of three seeds. The study therefore leaves the question of how branching and action conditioning separate unanswered at this budget. Answering it needs learned encoders that clear the untrained baseline first, and comparisons against stronger, external action-conditioned methods.

AI USE STATEMENT

An AI coding agent (Claude Code, Anthropic) carried out most of the work under instructions from the human project owner. It wrote the environment, accounting, training, evaluation, and export code and the unit tests, it ran all experiments reported here, and it drafted this manuscript. An AI sub-agent used web search and page fetching to check bibliographic metadata and passages of the cited papers; each reference’s verification level is recorded in `paper/claim_evidence.tsv`. **Status: HUMAN REVIEW PENDING.** At the time of this draft, no human author has reviewed the code, the results, or the text, and the draft must not be treated as reviewed or endorsed.

REPRODUCIBILITY STATEMENT

Every number in this draft is produced by `paper/export_results.py` from the raw run files listed in `paper/result_sources.json`; the export manifest records their SHA-256 hashes.

Each run directory contains its frozen configuration, a raw event log, per-cell results, and a manifest with the git commit, source and configuration hashes, transition ledgers, CPU time, multiply-accumulate counts, and peak memory. The implementation uses only the Python standard library. Appendix A gives the exact commands. The code and logs have not been publicly released.

REFERENCES

- André Barreto, Will Dabney, Rémi Munos, et al. Successor features for transfer in reinforcement learning. In *Advances in Neural Information Processing Systems 30*, 2017.
- Peter Dayan. Improving generalization for temporal difference learning: The successor representation. *Neural Computation*, 5(4):613–624, 1993.
- Carlton Downey, Ahmed Hefny, Byron Boots, Geoffrey J. Gordon, and Boyue Li. Predictive state recurrent neural networks. In *Advances in Neural Information Processing Systems 30*, 2017.
- Karol Gregor, Danilo Jimenez Rezende, Frederic Besse, Yan Wu, Hamza Merzic, and Aäron van den Oord. Shaping belief states with generative environment models for RL. In *Advances in Neural Information Processing Systems 32*, 2019.
- Christopher Grimm, André Barreto, Satinder Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems 33*, 2020.
- Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Bernardo A. Pires, and Rémi Munos. Neural predictive belief representations. *arXiv preprint arXiv:1811.06407*, 2018.
- Zhaohan Daniel Guo, Bernardo Avila Pires, Bilal Piot, et al. Bootstrap latent-predictive representations for multitask reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pp. 3875–3886, 2020.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, et al. Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *PMLR*, pp. 2555–2565, 2019.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024.
- Michael R. James and Satinder Singh. Learning and discovery of predictive state representations in dynamical systems with reset. In *Proceedings of the 21st International Conference on Machine Learning (ICML)*, 2004.
- Khimya Khetarpal, Zhaohan Daniel Guo, Bernardo Avila Pires, et al. A unifying framework for action-conditional self-predictive reinforcement learning. In *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 258 of *PMLR*, pp. 181–189, 2025.
- Jeongyeol Kwon, Liu Yang, Robert Nowak, and Josiah Hanna. An empirical study on the power of future prediction in partially observable environments. *arXiv preprint arXiv:2402.07102*, 2024. Version 2, March 2025.
- Michael L. Littman, Richard S. Sutton, and Satinder Singh. Predictive representations of state. In *Advances in Neural Information Processing Systems 14*, 2001.
- Andrew K. McCallum. *Reinforcement Learning with Selective Perception and Hidden State*. PhD thesis, University of Rochester, 1996.
- Zakaria Mhammedi, Dylan J. Foster, and Alexander Rakhlin. The power of resets in online reinforcement learning. In *Advances in Neural Information Processing Systems 37*, 2024.

- Tianwei Ni, Benjamin Eysenbach, and Ruslan Salakhutdinov. Recurrent model-free RL can be a strong baseline for many POMDPs. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022.
- Tianwei Ni, Michel Ma, Benjamin Eysenbach, and Pierre-Luc Bacon. When do transformers shine in RL? decoupling memory from credit assignment. In *Advances in Neural Information Processing Systems 36*, 2023.
- Tianwei Ni, Benjamin Eysenbach, Erfan Seyedsalehi, et al. Bridging state and history representations: Understanding self-predictive RL. In *International Conference on Learning Representations (ICLR)*, 2024.
- Junhyuk Oh, Xiaoxiao Guo, Honglak Lee, Richard Lewis, and Satinder Singh. Action-conditional video prediction using deep networks in Atari games. In *Advances in Neural Information Processing Systems 28*, 2015.
- Mohammad Reza Samsami, Artem Zhohus, Janarthanan Rajendran, and Sarath Chandar. Mastering memory tasks with world models. In *International Conference on Learning Representations (ICLR)*, 2024.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.
- John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015.
- Max Schwarzer, Ankesh Anand, Rishab Goel, R Devon Hjelm, Aaron Courville, and Philip Bachman. Data-efficient reinforcement learning with self-predictive representations. In *International Conference on Learning Representations (ICLR)*, 2021.
- Satinder Singh, Michael R. James, and Matthew R. Rudary. Predictive state representations: A new theory for modeling dynamical systems. In *Proceedings of the 20th Conference on Uncertainty in Artificial Intelligence (UAI)*, pp. 512–519, 2004.
- Jayakumar Subramanian, Amit Sinha, Raihan Seraj, and Aditya Mahajan. Approximate information state for approximate planning and reinforcement learning in partially observed systems. *Journal of Machine Learning Research*, 23(12):1–83, 2022.
- Richard S. Sutton and Brian Tanner. Temporal-difference networks. In *Advances in Neural Information Processing Systems 17*, 2004.
- Arun Venkatraman, Nicholas Rhinehart, Wen Sun, et al. Predictive-state decoders: Encoding the future into recurrent networks. In *Advances in Neural Information Processing Systems 30*, 2017.
- Steven D. Whitehead and Dana H. Ballard. Learning to perceive and act by trial and error. *Machine Learning*, 7:45–83, 1991.
- Amy Zhang, Rowan McAllister, Roberto Calandra, Yarin Gal, and Sergey Levine. Learning invariant representations for reinforcement learning without reconstruction. In *International Conference on Learning Representations (ICLR)*, 2021.

A REPRODUCTION COMMANDS AND TECHNICAL CHECKS

All code is pure Python 3.11 with no third-party packages. The commands below reproduce the reported runs, starting from the repository root.

```
python3 -m unittest discover -s tests -t . -v
OMP_NUM_THREADS=1 python3 -m acpr.diagnose \
    --config configs/diag_eval_discriminability.json --out runs/<id>
python3 paper/export_results.py
```

The 2×2 study was run with the following command:

```
OMP_NUM_THREADS=1 python3 -m acpr.study2x2 \
  --config configs/study_2x2_symmetric.json --out runs/<id>
```

The unit tests check the following technical properties. First, the aliasing construction, verified by exact enumeration. Second, transition accounting: the recorded main and branch steps equal the ledger totals, the cap is hard and spent exactly, and restores are counted. Third, information access: signatures, field whitelists, serialized datasets, and encoder causality. Fourth, split disjointness. Fifth, horizon semantics: termination padding versus truncation masking. Sixth, finite-difference gradient checks of the pure-Python autodiff. Seventh, positive and negative controls of the readout, and CPU-cap enforcement through `RLIMIT_CPU`. These are numeric fixture checks of the implementation. They are not proofs and not evidence of method efficacy.

B HISTORY OF PROTOCOL CHANGES

The protocol changed during development, and we record the changes here for transparency. After the first smoke run, the clue probe collapsed to the majority class, and planner metrics were dominated by the test set’s clue base rate. The probe features were therefore standardized, and test clues were balanced by the evaluator. After the second smoke run and a positive-control check, forced-commit accuracy replaced full-action success as the primary metric. The reason was that an under-trained head made the full-action planner stall at the junction. For the same reason, head training was lengthened. The evaluator diagnostic then added train-only standardization of encoder features before the head, applied identically to every encoder, and checkpoint selection among 20/40/80 epochs. None of these changes was made after observing learned-encoder comparisons. They were, however, made after looking at smoke and control outputs, so seeds used in those runs count as development data.

C ADDITIONAL 2×2 STUDY TABLES

Table 5: Per-seed experience ledgers of the 2×2 study (from the run manifest). Branch steps and restores occur only in the branching pretraining data. The readout pool and calibration are restore-free.

Seed	Data	Main steps	Branch steps	Restores	Episodes
1	Pretraining, branching	3,276	6,724	3,388	398
1	Pretraining, no branching	10,000	0	0	1,217
1	Readout pool, train	10,000	0	0	1,197
1	Readout pool, dev	2,000	0	0	243
1	Calibration (probe)	1,095	0	0	200
2	Pretraining, branching	3,266	6,734	3,356	393
2	Pretraining, no branching	10,000	0	0	1,201
2	Readout pool, train	10,000	0	0	1,201
2	Readout pool, dev	2,000	0	0	244
2	Calibration (probe)	1,112	0	0	200
3	Pretraining, branching	3,342	6,658	3,304	399
3	Pretraining, no branching	10,000	0	0	1,202
3	Readout pool, train	10,000	0	0	1,207
3	Readout pool, dev	2,000	0	0	240
3	Calibration (probe)	1,110	0	0	200

D PLANNED EXPERIMENTS (NOT RUN)

None of the following has been run. No result of any of them appears in this draft.

Table 6: Full-action planner metrics per seed. They are confounded by commit-versus-stall behaviour (Section 5.1) and are not used for decisions.

Arm	Full-action success	Mean return	Timeout	Forced ties
A: ACP + branching	0.500 / 0.500 / 0.105	0.000 / 0.000 / -0.045	0.000 / 0.000 / 0.745	0 / 0 / 0
B: ACP + no branching	0.500 / 0.000 / 0.000	0.000 / 0.000 / 0.000	0.000 / 1.000 / 1.000	0 / 0 / 0
C: MSP + branching	0.500 / 0.000 / 0.500	0.000 / 0.000 / 0.000	0.000 / 1.000 / 0.000	0 / 0 / 0
D: MSP + no branching	0.500 / 0.000 / 0.500	0.000 / 0.000 / 0.000	0.000 / 1.000 / 0.000	0 / 0 / 0
E: untrained recurrent	0.500 / 0.000 / 0.000	0.000 / 0.000 / 0.000	0.000 / 1.000 / 1.000	0 / 0 / 0
F: informative control	1.000 / 1.000 / 1.000	1.000 / 1.000 / 1.000	0.000 / 0.000 / 0.000	0 / 0 / 0

Table 7: Pretraining prediction metrics per seed, for description only. The ACP and MSP objectives are not like-for-like: ACP predicts futures conditioned on actions, MSP predicts the action-marginal future. Readout-development metrics are computed on the restore-free development pool.

Arm	Train loss, epoch 1	Train loss, final	Readout-dev obs. NLL	Readout-dev reward MSE
A: ACP + branching	2.200 / 2.160 / 2.143	1.199 / 1.227 / 1.237	1.157 / 1.142 / 1.150	0.119 / 0.122 / 0.119
B: ACP + no branching	1.726 / 1.669 / 1.711	1.237 / 1.245 / 1.255	1.152 / 1.140 / 1.121	0.120 / 0.132 / 0.123
C: MSP + branching	2.255 / 2.263 / 2.182	1.612 / 1.622 / 1.614	1.529 / 1.491 / 1.512	0.120 / 0.124 / 0.120
D: MSP + no branching	1.918 / 1.923 / 1.882	1.583 / 1.591 / 1.587	1.469 / 1.491 / 1.472	0.123 / 0.126 / 0.122

- **Diagnosing the learned-encoder floor.** Three kinds of run could separate the explanations listed as conjectures in Section 6. The first would pretrain for longer or with the pre-registered learning-rate grid. The second would vary the prediction horizon relative to corridor length. The third would give the readout head more capacity or a direct probe-based head. None was run. Any such run would be exploratory, would need new seeds, and must not be presented as a confirmation. Changing the environment to make an effect appear after seeing this negative result is excluded by our protocol.
- **Pre-registered pilot.** Ten seeds in both maze variants with a paired bootstrap analysis. It was not approved and not run. Seeds 1–3 of that plan have now been used by the 2×2 study and are development data.
- **Asymmetric control variant.** In this variant, action-marginal futures also carry the clue. It was not run.
- **Unstandardized readout.** The head without feature standardization, as in the original pilot plan. It was not run.
- **External comparisons.** An action-conditioned future-prediction method in the style of Kwon et al. (2024), recurrent world models, and recurrent model-free agents. None was run.
- **Other environments.** Longer corridors, other task families, and partially observable continuous control. None was run, and none is planned without a positive result here.